

Supervised Learning

Supervised Learning

Supervised learning is a type of machine learning where the model learns a mapping function $f: X \rightarrow Y$ from a labeled training dataset, i.e. a dataset in which every input (X) is paired with a known, correct output (Y). Once trained, the model uses this learned mapping to predict the output for new, unseen inputs.

Key components

- **Features (independent variables)** — the input variables (X) used to make a prediction.
- **Target / Label (dependent variable)** — the known output (Y) the model is trying to predict.
- **Training data** — the labeled (X, Y) pairs used to fit the model's parameters.
- **Loss / Cost function** — measures how far the model's predictions are from the actual values; the model is trained to minimize this.
- **Generalization** — the model's ability to perform well on new, unseen data, not just the data it was trained on.

Two broad types of supervised learning problems

Regression – the target variable is continuous / numeric (e.g. predicting house price, temperature, salary).

Classification – the target variable is categorical / discrete (e.g. predicting spam vs not-spam, disease vs no disease).

Dimensionality Reduction

Dimensionality reduction is the process of reducing the number of input features (dimensions) in a dataset while trying to preserve as much useful/important information as possible. This topic is only introduced at a basic level here — **PCA and ICA will be covered in detail in a separate dedicated session.**

Why do we need it?

- **Curse of dimensionality** — as the number of features grows, data becomes sparse and models need exponentially more data to generalize well.
- Removes redundant / highly correlated features that add noise rather than information.
- Speeds up training and reduces memory/storage requirements.
- Helps reduce overfitting by simplifying the model.
- Makes it possible to visualize high-dimensional data in 2D or 3D.

Two broad approaches

Feature Selection – choosing a subset of the original features (e.g. dropping features with low importance or high correlation with another feature), without transforming them.

Feature Extraction – transforming the original features into a new, smaller set of derived features that still capture most of the information (e.g. Principal Component Analysis (PCA), Independent Component Analysis (ICA)).

Train, Cross-Validation (CV) and Test Split

Before training a model, the dataset is split into separate parts so that we can fairly train, tune, and evaluate the model.

Training set – the portion of data used to actually fit / learn the model's parameters (e.g. coefficients, tree splits).

Validation / Cross-Validation (CV) set – used to tune hyperparameters and select between models, without touching the test set. A common technique is **k-fold cross-validation**, where the training data is split into k parts; the model is trained on (k-1) parts and validated on the remaining part, and this is repeated k times, rotating the validation fold.

Test set – a completely held-out portion of data used only once, at the very end, to get an unbiased estimate of how the final model will perform on real, unseen data.

Typical split ratios

- **Simple split:** 70% Train / 15% Validation / 15% Test, or 80% Train / 20% Test with CV done inside the training portion.

Supervised Regression

Linear Regression

Linear regression models the relationship between one or more independent variables (X) and a continuous dependent variable (Y) by fitting a straight line $Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$.

Key assumptions: linearity between X and Y, independence of errors, constant variance of errors (homoscedasticity), normally distributed residuals, and no strong multicollinearity among predictors.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import numpy as np

X = np.array([[1], [2], [3], [4], [5]]) # feature
y = np.array([2, 4, 6, 8, 10])         # target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)             # learn slope & intercept
predictions = model.predict(X_test)
print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)
print("Predictions:", predictions)
```

Polynomial Regression

Polynomial regression is an extension of linear regression used when the relationship between X and Y is curved rather than a straight line.

- It models Y as an nth-degree polynomial of X, **e.g. $Y = b_0 + b_1X + b_2X^2 + b_3X^3 + \dots$**

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
import numpy as np
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([1, 4, 9, 16, 25])    # curved (quadratic)
relationship
# transform X into [X, X^2] then fit linear regression on
top
poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
poly_model.fit(X, y)
predictions = poly_model.predict(X)
print("Predictions:", predictions)
```

Minute code change: The only real change from plain Linear Regression is wrapping the model with `PolynomialFeatures(degree=n)` in a pipeline — the `fit()`, `predict()` calls and overall workflow stay exactly the same.

KNN Regression

KNN Regression works by finding the **K closest data points (neighbors)** to a new input and predicting the output as the **average (or weighted average)** of their target values.

How It Works

1. Choose a value for **K** (number of neighbors).
2. Calculate the distance between the new data point and all training data points (commonly using **Euclidean distance**).
3. Select the **K nearest neighbors**.
4. Compute the prediction:
 - **Simple average** of neighbors' target values, or
 - **Weighted average**, where closer neighbors have greater influence.

Follow these steps in code

Step 1: split data into training and test sets

Step 2: create the KNN Regressor

n_neighbors=3 means the prediction for a new point is based on the average of its 3 nearest neighbors' target values

Step 3: fit the model

KNN is a "lazy learner" - fit() just stores the training data, no actual weights/coefficients are learned at this stage

Step 4: predict on test data

For each test point, the model: **a) computes distance** (default: Euclidean) to every training point

b) picks the 3 closest training points (n_neighbors=3)

c) returns the AVERAGE of their target (y) values as the prediction

Step 5 (optional): evaluate performance using a regression metric

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import train_test_split
import numpy as np

# Sample data: single feature X, continuous target y
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8]])
```

```
y = np.array([1.5, 2.0, 3.5, 4.0, 5.5, 6.0, 7.5, 8.0])  
# step 1  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=42  
)  
#step 2  
knn_reg = KNeighborsRegressor(n_neighbors=3)  
# step 3  
knn_reg.fit(X_train, y_train)  
# step 4  
predictions = knn_reg.predict(X_test)  
print("Test inputs:", X_test.ravel())  
print("Predicted values:", predictions)  
print("Actual values:", y_test)  
# step 5 (optional) to find error  
from sklearn.metrics import mean_squared_error  
mse = mean_squared_error(y_test, predictions)  
print("Mean Squared Error:", mse)
```

Multicollinearity

Multicollinearity occurs when two or more independent variables in a regression model are highly correlated with each other. This makes it difficult for the model to isolate the individual effect of each predictor on the target, and causes the estimated coefficients to become

unstable and hard to interpret (small changes in data can cause large swings in coefficient values).

Regularized Regression

Regularization adds a penalty term to the linear regression loss function to discourage the model from assigning very large weights to any feature. This reduces overfitting and helps handle multicollinearity, at the cost of introducing a small amount of bias.

Ridge Regression (L2 regularization) — adds a penalty equal to $\lambda \times (\text{sum of squared coefficients})$ to the loss function. It shrinks coefficients towards zero, but never makes them exactly zero, so all features are retained, just with reduced influence. Useful when many features are relevant but correlated.

Lasso Regression (L1 regularization) — adds a penalty equal to $\lambda \times (\text{sum of absolute values of coefficients})$ to the loss function. Unlike Ridge, Lasso can shrink some coefficients exactly to zero, effectively performing automatic feature selection.

In both cases, λ (lambda) is a hyperparameter that controls the strength of the penalty — $\lambda = 0$ reduces to plain linear regression, while a very large λ forces coefficients close to (or exactly) zero.

Support Vector Regression (SVR)

SVR is the regression version of the Support Vector Machine (SVM) algorithm. Instead of trying to minimize the error for every single point, SVR tries to fit a function such that most data points fall within a certain margin (called epsilon, ϵ) around the predicted line/curve. Points inside this ϵ -tube contribute zero loss (this is called the epsilon-insensitive loss), while only points outside the tube (the "support vectors") influence the final model. SVR can use kernels (linear, polynomial, RBF) to model non-linear relationships as well.

Classifier-KNN-DecisionTree

Supervised Classification

KNN Classifier

The K-Nearest Neighbors (KNN) classifier predicts the class label of a new data point by looking at the 'k' closest points in the training data and assigning the majority class among them (a vote), instead of averaging numeric values.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
import numpy as np

# Sample data: single feature X, categorical target y (0 or 1)
X = np.array([[1], [2], [3], [6], [7], [8]])
y = np.array([0, 0, 0, 1, 1, 1])    # class labels, not
continuous numbers
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42
)

# n_neighbors=3: look at the 3 nearest points
knn_clf = KNeighborsClassifier(n_neighbors=3)
knn_clf.fit(X_train, y_train)

# For each test point, KNN finds the 3 nearest training
points
# and assigns the class that appears MOST OFTEN among
```

them (majority vote)

```
predictions = knn_clf.predict(X_test)
print("Predicted classes:", predictions)
print("Actual classes:", y_test)
from sklearn.metrics import accuracy_score
print("Accuracy:", accuracy_score(y_test, predictions))
```

How is KNN Classifier different from KNN Regression?

Both use the exact same core idea — find the 'k' nearest data points to a query point using a distance metric (usually Euclidean). The difference lies entirely in what is done with those neighbors and what type of output is produced:

Aspect	KNN Regression	KNN Classifier
Target variable	Continuous / numeric	Categorical / discrete class
Aggregation of neighbors	Average (or weighted average) of neighbors' target values	Majority vote among neighbors' class labels
Output	A predicted number (e.g. 5.7)	A predicted class label (e.g. 'spam')
sklearn class	KNeighborsRegressor	KNeighborsClassifier

Aspect	KNN Regression	KNN Classifier
Evaluation metric	MSE, RMSE, R ²	Accuracy, Precision, Recall, F1-score

Decision Trees

A decision tree splits the dataset repeatedly based on feature values, forming a tree of if-else decision rules, until it reaches 'leaf' nodes that give a final prediction.

At each split, the algorithm chooses the feature and threshold that best separates the classes.

Why / when do we use it?

Decision trees are easy to interpret and visualize, handle both numeric and categorical features, and require little data preprocessing (e.g. no feature scaling needed).

Their main **drawback** is a tendency to overfit if grown too deep, which is why they are often limited via **max_depth** or combined into ensembles like **Random Forests**.

Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np
# Sample data
```

```
X = np.array([[25, 40000], [45, 80000], [35, 60000],
              [50, 90000], [23, 20000], [40, 75000]])
y = np.array([0, 1, 0, 1, 0, 1]) # e.g. 0 = did not buy, 1 =
bought
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42
)
# max_depth limits how deep the tree can grow, to reduce
overfitting
tree_clf = DecisionTreeClassifier(max_depth=3, criterion="gini",
random_state=42)
tree_clf.fit(X_train, y_train)
predictions = tree_clf.predict(X_test)
print("Predicted classes:", predictions)
print("Actual classes:", y_test)
print("Accuracy:", accuracy_score(y_test, predictions))
# feature_importances_ shows which feature the tree
relied on most
print("Feature importances:", tree_clf.feature_importances_)
```